BRAUDE
College of Engineering, Karmiel

Software Engineering Department
Braude College of Engineering
Final Project - Phase B (61999)

ActiveSaga : A VR Sport Game

26-1-D-15

Alexandra Belkind
Daniel Chernov

Supervisor: Dr. Moshe Sulamy

GitHub-ActiveSaga

Daniel Chernov: Daniel.Chernov@e.braude.ac.il
Alexandra Belkind: Alexandra.Belkind@e.braude.ac.il
Supervisor: Moshe Sulamy: MosheSu@braude.ac.il

Contents

Abstract

ActiveSaga is a standalone Virtual Reality fitness game developed for the Meta Quest platform. The project was designed to make home workouts more engaging by turning physical exercise into interactive gameplay. Instead of following a traditional workout routine, the player enters a fantasy-inspired VR environment where real body movements control the game.

The system includes two playable modes: Run Game and Fight Game. In the Run Game, the player runs in place, jumps, squats, dodges, and reacts to obstacles while progressing through the environment. In the Fight Game, the player attacks enemies using the controllers, dodges incoming threats, jumps, and reacts to waves of enemies. The system detects the player's physical actions in real time using only the VR headset and controllers, without external body sensors.

To support long-term motivation, ActiveSaga includes XP, coins, levels, daily missions, weekly activity rewards, help screens, pause functionality, audio feedback, and end-game results. Player data is saved through a backend server implemented with Node.js and Express, connected to a MongoDB database. In the final version, the Meta Quest APK communicates with the deployed backend server on Render, allowing user authentication, progress loading, session result submission, and database synchronization.

The final result is a working VR fitness game that runs as a standalone APK on the Meta Quest headset. The project combines VR development, real-time movement recognition, gameplay design, backend communication, authentication, database persistence, deployment, and user-centered testing.

1. Introduction and Motivation

Physical inactivity and repetitive home workouts make it difficult for many people to stay motivated over time [9]. Although fitness applications and workout videos are widely available, they often lack the engagement and feedback needed to encourage consistent activity.

Virtual Reality offers a more interactive approach by placing the user inside a 3D environment where physical movement becomes part of the experience. ActiveSaga was developed to use this advantage and turn exercise into gameplay. The player runs, jumps, squats, dodges, and fights inside a fantasy-inspired VR environment while progressing through the game.

The target users of ActiveSaga are people who want to perform physical activity at home in a more engaging way, especially users who enjoy games, own a Meta Quest headset, and prefer a workout experience that feels less repetitive than traditional exercise.

The project focuses on three main goals: creating an immersive VR experience that requires real physical movement, detecting basic exercise actions using only standard VR hardware, and encouraging repeated use through XP, coins, levels, daily missions, weekly progress, and saved player data.

The final system includes two main game modes. The Run Game focuses on cardio movement, while the Fight Game focuses on combat, reaction, and body control. The client side was developed in Unity for Meta Quest, and the backend was implemented with Node.js, Express, and MongoDB to support authentication, player progress, missions, rewards, and game-session results.

ActiveSaga demonstrates how VR can combine physical movement, interactive gameplay, and long-term motivation into one complete software system.

2. Final System Overview

ActiveSaga is a standalone VR fitness game developed for the Meta Quest headset. The system combines physical exercise with immersive gameplay in order to make home workouts more engaging, interactive, and motivating.

The player interacts with the game inside a VR environment and controls the gameplay through real body movements. The system detects actions such as running in place, jumping, squatting, dodging, and attacking using the headset and controllers. These actions are translated into gameplay mechanics in real time.

The final system includes two playable game modes: Run Game and Fight Game. Each mode provides a different type of physical activity and gameplay experience. In addition, the game includes login and registration, player progress, daily missions, weekly activity rewards, help screens, pause functionality, audio feedback, and end-game results.

2.1 User Flow

The player starts ActiveSaga directly from the Meta Quest headset. The first screen is a login and registration screen displayed inside the VR environment. New users can create an account, while existing users can log in and continue with their saved progress.

After logging in, the player reaches the main menu. From this menu, the player can view progress information, daily missions, weekly activity status, and choose a game mode and difficulty level.

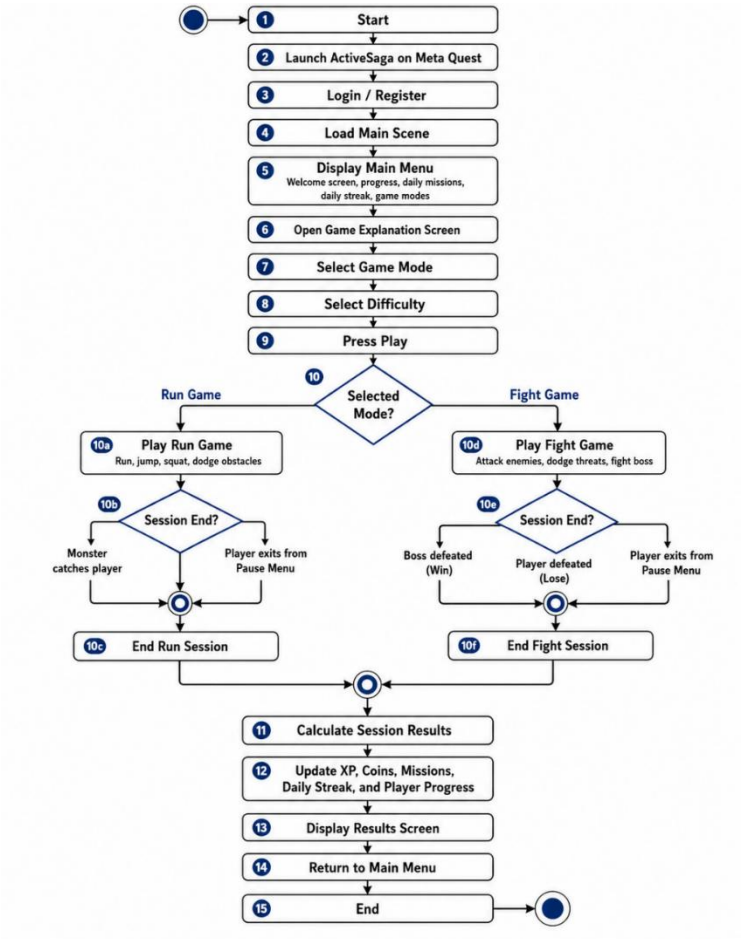


Figure 1: General User Flow Activity Diagram

The diagram presents the main user flow of ActiveSaga, from launching the game and logging in, through selecting a game mode and difficulty level, completing a gameplay session, viewing the results, and returning to the main menu

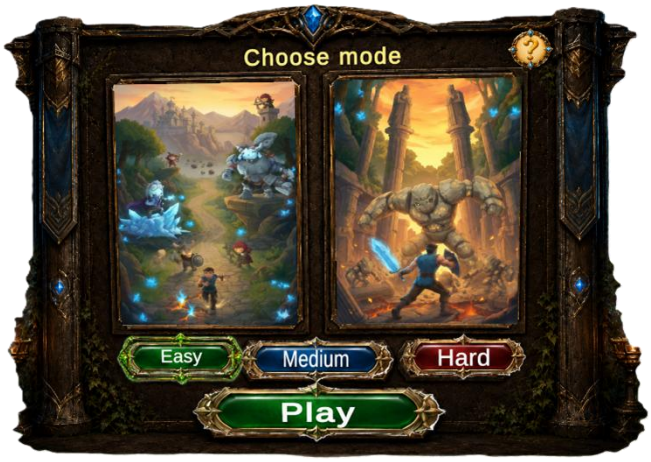
2.2 Game Modes

ActiveSaga includes two main game modes: Run Game and Fight Game.

The Run Game is focused on cardio activity. The player runs in place in order to move forward inside the virtual environment. During the session, the player must react to obstacles by jumping, squatting, or dodging. The mode is designed as an endurance challenge that encourages the player to stay active and improve personal performance over time.

The Fight Game is focused on combat, reaction, and body control. In this mode, the player faces a boss-like enemy that sends waves of attacking enemies. The player must attack enemies, avoid incoming threats, and survive until the boss is defeated or the session ends.

Both modes support three difficulty levels: easy, medium, and hard. The difficulty level affects the challenge of the session and is also connected to the daily mission system.



2.3 Movement-Based Interaction

A central part of ActiveSaga is movement-based interaction. Instead of using only buttons or traditional controls, the player's physical actions become part of the gameplay.

The system detects the following main movements:

- Running in place
- Jumping
- Squatting
- Dodging
- Attacking with controllers

This approach makes the game more active and immersive. It also allows the player to exercise while progressing through the game.

The movement recognition is performed using only standard Meta Quest hardware. No external leg trackers, cameras, or wearable sensors are required.

2.4 Progression and Motivation

ActiveSaga uses gamification to encourage repeated physical activity [10]. After each session, the player receives feedback and rewards based on performance.

The progression system includes:

- XP
- Coins
- Levels
- Daily missions
- Weekly progress
- End-game results

Daily missions give the player short and focused goals, such as running a specific distance or defeating enemies. These missions must be completed within the same gameplay session.

The weekly progress system encourages consistency. To complete a daily activity requirement, the player must play for at least five minutes during that day. The five minutes can be accumulated across several sessions. Completing five active days in the same week grants a larger weekly reward.

2.5 Results and Saved Progress

At the end of each gameplay session, ActiveSaga displays a results screen. This screen gives the player immediate feedback and shows the rewards earned during the session.

The results screen includes information such as:

- Time played
- XP earned
- Coins earned
- Current level
- XP progress
- Distance
- Enemies defeated
- Jumps performed

After the session ends, the player's progress is saved through the backend server and MongoDB database. This allows the player to continue from the same state in future sessions.

2.6 System Summary

The final ActiveSaga system includes:

- A standalone Unity VR client for Meta Quest
- Two playable game modes
- Real-time movement-based interaction
- VR-friendly menus and help screens
- XP, coins, levels, daily missions, and weekly rewards
- End-game results and saved player progress
- Backend synchronization with MongoDB

Together, these components create a complete VR fitness game that combines physical activity, interactive gameplay, and long-term motivation.

3. System Architecture and Implementation

ActiveSaga uses a client-server architecture. The Unity VR client runs as a standalone application on the Meta Quest headset [2] and handles all real-time gameplay, including movement detection, player actions, UI, audio, and game logic.

The backend, implemented with Node.js and Express, is connected to MongoDB and manages persistent data such as authentication, player profiles, XP, coins, levels, daily missions, weekly progress, and saved statistics.

This separation keeps the VR experience fast and responsive while ensuring that player progress is stored and updated consistently.

3.1 High-Level Architecture

The system is divided into several main layers:

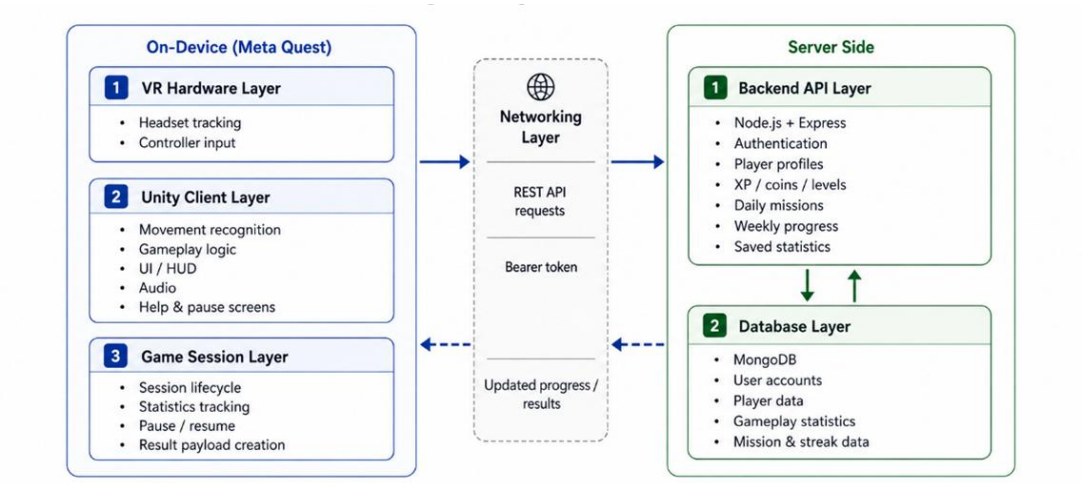


Figure 2: High-Level System Architecture
The diagram presents the main architectural layers of ActiveSaga, including the Meta Quest client, Unity gameplay layers, networking layer, backend API, and MongoDB database

VR Hardware Layer
The Meta Quest headset and controllers provide tracking data. The headset position is used for movement recognition, while the controllers are used for menu interaction and gameplay actions.

Unity Client Layer
The Unity client includes the game scenes, movement recognition, gameplay logic, UI panels, HUD, audio, pause menu, help screens, and results screens. This layer runs directly on the headset.

Game Session Layer
This layer manages each gameplay session from start to finish. It tracks active play time, collects session statistics, handles pause and resume, creates the final session result, and sends it to the backend.

Networking Layer
The Unity client communicates with the backend using REST API requests. Authenticated requests include a Bearer token that identifies the logged-in player.

Backend API Layer
The backend receives requests from Unity, validates authentication, processes game results, calculates rewards, updates missions and weekly progress, and returns updated data to the client.

Database Layer
MongoDB stores user accounts, player profiles, XP, coins, levels, daily missions, weekly progress, and gameplay statistics.

Together, these layers create a complete system where the headset manages the active VR experience and the backend manages persistent player progress.

3.2 Unity Client Implementation

The Unity client was developed using Unity and C# [1]. It contains the systems that must run locally and respond immediately during gameplay. This includes movement recognition, player locomotion, collision handling, enemy behavior, UI interaction, audio feedback, pause control, help screens, and result display.

A component-based structure was used to keep the project organized. For example, movement analysis is separated from gameplay behavior. The movement recognition components detect physical actions, while other systems decide how those actions affect the game. This makes the system easier to test, tune, and maintain.

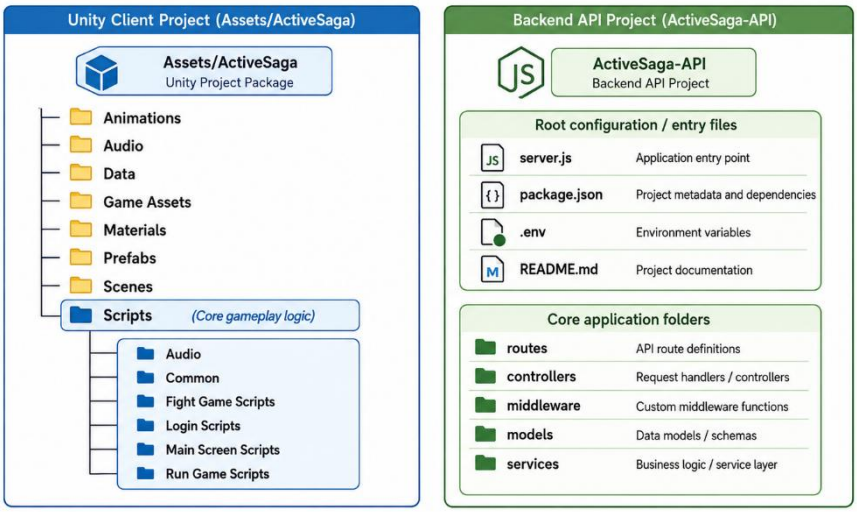


Figure 3: Project Package Structure Diagram
The diagram presents the main software package structure of ActiveSaga, including the Unity client folders, gameplay script modules, backend API folders, configuration files, and main server entry point

The main real-time client responsibilities are:

- Reading headset and controller tracking data
- Detecting running, jumping, squatting, dodging, and attacks
- Managing Run Game and Fight Game gameplay logic
- Tracking session statistics
- Displaying HUD, menus, help screens, pause menu, and results
- Sending the final session result to the backend

This design allows the game to remain responsive on the headset without depending on network communication during active gameplay.

3.3 Movement Recognition Implementation

Before gameplay starts, the system performs a height calibration step. During this step, the player stands naturally, and the headset Y position is saved as the player's BaseHeight. This calibrated value provides a stable reference for movements that depend on body height, mainly squatting and jump validation.

The movement recognition system is implemented through separate analyzer components. Each analyzer reads headset or controller tracking data and converts it into gameplay input, such as run intensity, squat state, jump events, dodge reactions, or attack actions. This separation keeps the detection logic organized and allows each movement type to use the most suitable input data and filtering method.

Not all movements use the calibrated height directly. Running is based on repeated vertical headset motion over time, squatting is based on the player's height relative to the calibrated BaseHeight, and jumping combines upward velocity with height-based validation. Dodging and attacking are detected from movement patterns during gameplay, using headset/body movement and controller velocity.

Movement	Input Data	Detection Logic	Main Challenge	Implemented Solution
Running in place	Headset Y position between frames	The system compares the current headset height to the previous frame and calculates the absolute vertical velocity. Valid repeated vertical movement is converted into a run intensity value between 0 and 1	Small natural head movements, breathing, or very strong vertical movement may be confused with running	Movements below 0.15 m/s are ignored as noise. Movements above 1.2 m/s are treated as too strong for running and do not create new running input. The final run intensity is smoothed before being sent to the locomotion system
Squatting	Current headset height compared to calibrated BaseHeight	The system compares the current headset height to the calibrated BaseHeight. If the headset drops below BaseHeight - 0.30m, the player enters the squat state. The player exits the squat state only after rising above BaseHeight - 0.10m	Detection accuracy depends on correct height calibration, and small height changes near the threshold may cause unstable detection	The system uses height calibration and two different thresholds: one for entering squat and one for exiting squat. This hysteresis prevents repeated triggering and cancelling around a single threshold
Jumping	Upward headset movement and calibrated BaseHeight	The system calculates the upward vertical velocity of the headset using the height change between frames. If the upward velocity is greater than 1.8 m/s, the movement is considered a jump candidate	Standing up quickly after a squat can create a high upward velocity and may look like a jump.	Before confirming the jump, the system checks the calibrated BaseHeight. If the headset is still below BaseHeight - 0.30m, the movement is ignored as squat recovery. A 0.8 second cooldown prevents one jump from being counted multiple times.

This approach keeps the system accessible because it works with standard VR hardware only, while still allowing real physical movement to control the gameplay.

3.4 Backend, Database and Security

The backend was implemented using Node.js and Express [3]. It exposes REST API endpoints that allow the Unity client to register users, log in, load player data, retrieve missions, retrieve weekly progress, and submit completed gameplay sessions.

MongoDB is used to store persistent player data [4]. This includes user accounts, player profiles, XP, coins, levels, total play time, total distance, daily missions, weekly progress, and gameplay statistics.

Authentication is handled using JWT tokens [6]. After login or registration, the backend returns a token to the Unity client. The client sends this token in the Authorization header when accessing protected player routes [6]. Passwords are not stored as plain text; they are hashed before being saved in the database using bcrypt [7].

The Unity client does not access MongoDB directly. All database operations pass through the backend, which validates requests, calculates rewards, updates missions and progress, and saves the final data.

This structure keeps sensitive logic on the server side and prevents uncontrolled updates to the database.

3.5 Session Result Flow

During gameplay, Unity tracks the session locally. This includes play time, movement actions, enemies defeated, jumps, distance, dodges, waves, and other relevant statistics according to the selected mode.

When the session ends, Unity creates a result payload and sends it to the backend. The backend validates the player token, processes the session data, calculates XP and coins, updates the player profile, checks daily mission completion, updates weekly progress, and saves the changes in MongoDB.

The backend then returns the updated result to Unity, and the player sees the final results screen.

The session result flow is presented in Figure 4.

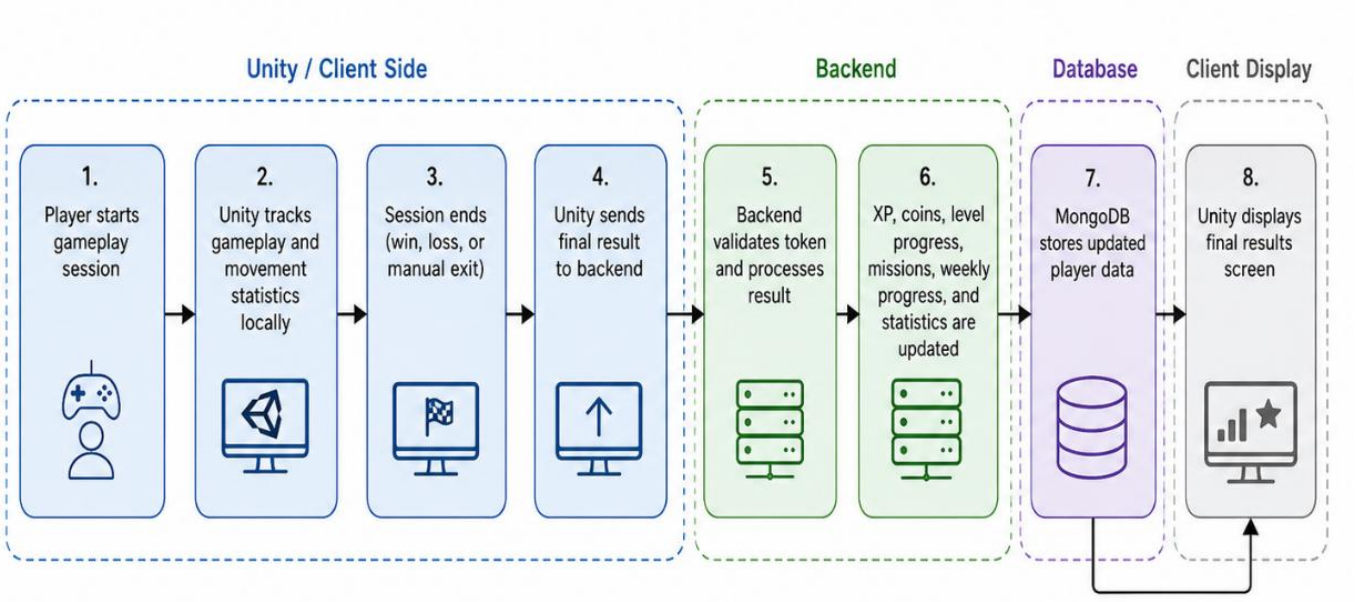


Figure 4: Session Result Flow Diagram

The diagram presents the session result flow, including local gameplay tracking in Unity, result submission to the backend, progress updates in MongoDB, and final results display on the client side

3.6 Tools and Technologies

The project was developed using the following main tools and technologies:

Tool / Technology	Purpose
Unity	VR client and game development
C#	Gameplay logic and movement recognition
Meta Quest	Target VR headset and testing device
Node.js	Backend runtime
Express	REST API server framework
MongoDB	Database for player data and progress
Mongoose	MongoDB data modeling
JWT	Token-based authentication
bcrypt	Password hashing
Render	Backend deployment
Git / GitHub	Version control and project repository
REST API	Communication between Unity and backend

3.7 Architecture Summary

The architecture of ActiveSaga supports the main goals of the project: responsive VR gameplay, real-time movement recognition, persistent player progress, and long-term motivation.

Unity handles the active VR experience on the headset, while the backend and database manage authentication, rewards, missions, weekly progress, and saved statistics. This division creates a clear and maintainable system that can be expanded in future versions.

4. Development Process and Challenges

The development of ActiveSaga was performed in an iterative and incremental manner. Since the project combines VR gameplay, physical movement recognition, backend communication, database persistence, and user interface design, the work was divided into smaller modules. Each module was developed, tested, improved, and then integrated into the full system.

The team first focused on the core VR prototype and movement recognition, because these parts define the main value of the project. After the basic gameplay was working, additional systems were added, including progression, daily missions, weekly rewards, backend synchronization, help screens, audio feedback, and the final standalone APK build.

The main development stages were:

1. Creating the initial Unity VR project and testing it on the Meta Quest headset.
2. Implementing movement recognition for running, jumping, and squatting.
3. Developing the Run Game and Fight Game gameplay mechanics.
4. Building the VR interface, including menus, HUD, pause screen, help screens, and results screen.
5. Implementing progression features such as XP, coins, levels, daily missions, and weekly rewards.
6. Developing the backend server and connecting it to MongoDB.
7. Connecting the Unity client to the backend through REST API requests.
8. Testing the full system on the headset and improving detected issues.
9. Deploying the backend and creating the final standalone Meta Quest build.

4.1 Main Development Challenges

During development, several technical and design challenges affected the final solution.

The first major challenge was movement recognition without external sensors. Since the system uses only the Meta Quest headset and controllers, lower-body actions such as running in place, jumping, and squatting had to be inferred from headset movement. This required calibration, threshold tuning, and repeated testing on the actual headset.

The second challenge was avoiding false movement detections. Small head movements could sometimes look like running, and standing up from a squat could look like a jump. To solve this, the movement logic was improved with filtering, height validation, and cooldown conditions.

The third challenge was VR user interface interaction. In VR, the player interacts with menus using controllers inside a 3D space, not with a mouse. Some buttons were not detected reliably at first, so the interaction areas were adjusted and the UI was kept simple, readable, and more forgiving.

The fourth challenge was deciding how to divide responsibility between Unity and the backend. Real-time gameplay had to remain local on the headset to avoid latency, while persistent data such as XP, coins, missions, and weekly progress had to be managed by the backend. The final solution was to send only the completed session result to the server after gameplay ends.

The fifth challenge was scope management. Several additional ideas were considered, such as a full shop system, cosmetic items, leaderboards, and advanced upgrades. However, the team decided to focus on completing and stabilizing the core features: two playable game modes, movement recognition, progression, daily missions, weekly rewards, backend synchronization, and a standalone VR build.

4.2 Design Decisions

Several important design decisions were made during the project.

The first decision was to use standard Meta Quest hardware only, without external body sensors. This made the system more accessible and easier to demonstrate, even though it required more work on movement recognition.

The second decision was to keep real-time gameplay logic on the Unity client. Movement detection, collisions, attacks, enemy behavior, audio feedback, and UI interaction must respond immediately, so they are handled locally on the headset.

The third decision was to place progression and saved data on the backend. XP, coins, levels, daily missions, weekly progress, and statistics are calculated and stored through the server and MongoDB. This keeps player progress consistent and easier to maintain.

The fourth decision was to make daily missions session-based. A daily mission must be completed within one gameplay session, which encourages focused physical effort. In contrast, weekly progress is based on accumulated daily play time, allowing more flexibility.

The fifth decision was to keep the final scope realistic. Instead of adding many unfinished features, the team focused on the features that best support the project goal: turning physical activity into a motivating VR game experience.

4.3 Development Summary

The development process showed that VR fitness games require a balance between gameplay, physical comfort, responsiveness, and reliable movement recognition. Many issues could only be discovered by testing on the actual Meta Quest headset.

The final system was shaped by practical engineering decisions: modular Unity components, local real-time gameplay, backend-based progression, repeated headset testing, and careful scope management.

These decisions helped ActiveSaga become a complete and demonstrable VR fitness game with two playable modes, movement-based interaction, persistent progress, and long-term motivation features.

5. Testing and Evaluation

The testing process of ActiveSaga focused on verifying that the system works as a complete VR fitness game. Since the project combines physical movement, VR interaction, gameplay logic, backend communication, database persistence, and progression systems, the tests were performed both on individual features and on the complete system.

In addition to functional correctness, the evaluation considered whether the progression and motivation features were clear to users and supported the intended loop of short repeated play sessions.

Most of the testing was performed manually on the Meta Quest headset. This was necessary because several important parts of the project cannot be fully evaluated inside the Unity Editor. Movement recognition, controller interaction, UI readability, physical comfort, and gameplay responsiveness must be tested while wearing the headset and using the system in real conditions.

The main testing goals were:

- Verify that the game runs as a standalone APK on the Meta Quest headset
- Verify that running, jumping, and squatting are detected correctly
- Verify that both Run Game and Fight Game are playable from start to finish
- Verify that the VR user interface is clear and usable
- Verify that login, registration, and backend synchronization work correctly
- Verify that XP, coins, levels, daily missions, weekly progress, and results are updated correctly
- Verify that the motivational features are understandable and support short repeated play sessions

5.1 Main Test Areas

The system was tested in several main areas.

Movement Recognition
Running, jumping, and squatting were tested repeatedly on the Meta Quest headset. The goal was to verify that the system detects real physical actions and reduces false detections, such as normal head movement being detected as running or standing up from a squat being detected as a jump. These issues were improved by tuning thresholds, adding filtering conditions, and using cooldown logic.

Gameplay Modes
Both Run Game and Fight Game were tested as complete gameplay sessions. The tests verified that the player can start a session, perform the required actions, interact with obstacles or enemies, complete or exit the session, and reach the results screen.

VR User Interface
The login screen, main menu, game mode selection, daily missions, weekly progress, help screens, pause menu, and results screen were tested inside the headset. Some VR button interaction issues appeared during development, so the interaction areas were adjusted to make selection easier and more reliable.

Backend and Persistence
Registration, login, profile loading, game-session submission, and database persistence were tested. The system was also tested after restarting the game to verify that saved progress could still be loaded from MongoDB.

Progression and Motivation Features

XP, coins, levels, daily missions, weekly progress, rewards, results feedback, sound feedback, and help screens were tested as part of the gameplay flow. The goal was to verify that users could understand how short repeated sessions contribute to progress and rewards. This testing did not measure long-term retention, but it confirmed that the motivational loop was visible and understandable to users.

5.2 Summary of Test Results

Test Area	Test Steps	Expected Result	Actual Result	Status
APK Build and Launch	Build the Unity project as a standalone APK and run it on the Meta Quest headset	The game should launch directly on the headset without requiring a PC connection	The game launched successfully as a standalone VR application	Passed
Login and Registration	Open the application, register a new user, and log in with an existing user	The system should create new accounts, authenticate existing users, and load the main menu after login	Registration and login worked correctly, and the player profile was loaded after authentication	Passed
Player Profile Loading	Log in, exit the application, reopen it, and log in again	Saved player data should remain available after restarting the game	The player profile and progress were loaded from the database after restarting the application	Passed
Run Game Flow	Start the Run Game, run in place, react to obstacles, and finish the session	The player should move forward, interact with obstacles, collect session statistics, and reach the results screen	The full Run Game session was playable from start to finish, including movement tracking and result display	Passed
Fight Game Flow	Start the Fight Game, attack enemies, avoid incoming threats, and complete or exit the session	The player should be able to attack, dodge, survive enemy waves, and reach the results screen	The Fight Game worked as a complete gameplay session, including attacks, enemy waves, health logic, and session ending	Passed
Running Detection	Perform running-in-place movements during the Run Game	The system should detect repeated physical movement and translate it into forward movement	Running was detected during gameplay after threshold tuning.	Passed after tuning
Squat Detection	Perform a squat when required by an obstacle or gameplay event	The system should detect a significant decrease in headset height compared to the calibrated standing height	Squat detection worked correctly after calibration and testing on the headset	Passed
Jump Detection	Perform jump actions during gameplay and	Real jumps should be detected, while	Jump detection worked after adding validation	Passed after tuning

	also test standing up quickly after a squat	standing up from a squat should not be incorrectly counted as a jump	and cooldown logic to reduce false detections	
VR UI Interaction	Use the controllers to press buttons in the login screen, main menu, mode selection, help screens, pause menu, and results screen	Buttons should be easy to select and should respond reliably in VR	Some interaction issues appeared during development, and the effective button areas were adjusted to improve reliability	Passed after adjustment
Pause and Resume	Start a gameplay session, open the pause menu, resume the game, and exit manually	The game should pause gameplay correctly, resume without losing session state, and allow manual exit	Pause, resume, and manual exit worked correctly during headset testing	Passed
Help Screens	Open the help screens from the relevant menus	The player should receive clear explanations about game modes, missions, weekly rewards, and controls	The help screens were displayed correctly and supported the user flow	Passed
Results Screen	Finish a gameplay session and review the end-game results	The results screen should display session statistics, rewards, and progress information	The results screen displayed the relevant session data and reward feedback	Passed
Backend Synchronization	Complete a gameplay session and submit the results to the backend	The backend should receive the session result, update the player profile, and return updated progress	Session results were submitted successfully, and the updated data was saved in MongoDB	Passed
Daily Missions	Complete a gameplay session that matches a daily mission goal	The mission should be completed only when the required goal is achieved in the same session	Daily mission completion worked according to the session-based mission logic	Passed
Weekly Progress	Play during the day and check the weekly activity progress	Daily activity should update the weekly progress system and contribute toward the weekly reward	Weekly progress was updated correctly after valid gameplay activity	Passed
Motivational Feedback	Complete a gameplay session and review rewards, XP, coins, level progress, and results feedback	The player should receive clear feedback that connects gameplay performance to	The results screen and progress systems provided immediate feedback after the session	Passed

		progress and rewards		
Progression Loop Clarity	Play short sessions and review daily activity, daily missions, and weekly progress	The system should clearly show that short gameplay sessions contribute to daily and weekly progress	Daily missions and weekly progress were displayed and updated correctly, including the five-minute daily activity requirement	Passed

5.3 User Testing

User testing was performed by the project team and by seven informal testers: six family members and one friend. All testers used the Meta Quest headset and experienced the game in real VR conditions.

The purpose of the testing was to check whether the game was understandable, comfortable, and usable for people who were not involved in the development process. The testers were asked to play the game, interact with the VR interface, try the main actions, and give feedback about the clarity and comfort of the experience.

At the beginning, the main issue was that the user interface was not clear enough for new users. Some users needed additional guidance to understand the game flow, missions, and available actions. Based on this feedback, the UI was improved and help explanations were added.

After these improvements, users were able to understand the main flow more easily and interact with the system using the headset and controllers. This helped validate the usability of the interface, help screens, game flow, and basic gameplay interaction.

The motivational aspect was not formally tested over time, because motivation and habit formation require repeated use across several days or weeks. However, users were able to understand the progression elements, such as XP, coins, daily missions, weekly progress, and results feedback. Future evaluation could test long-term motivation by asking users to play regularly over a longer period.

5.4 Evaluation Summary

The testing process showed that ActiveSaga achieved its main functional goals. The final system runs as a standalone VR application on the Meta Quest headset, detects the main physical movements, supports two playable game modes, saves player progress through the backend, and provides motivation through XP, coins, daily missions, weekly progress, and results.

The most important evaluation results are:

- The game runs independently on the Meta Quest headset.
- Running, jumping, and squatting are detected during gameplay.
- Both Run Game and Fight Game are playable.
- The player can register, log in, and continue with saved progress.
- Game results are submitted to the backend and saved in MongoDB.
- XP, coins, levels, daily missions, and weekly progress update correctly.
- The VR interface became more usable after controller interaction adjustments.

Overall, the tests confirmed that ActiveSaga is a working VR fitness game that combines physical activity, gameplay, progression, and persistent player data.

6. Results, Conclusions and Future Work

6.1 Project Results

The final result of the project is a working standalone VR fitness game called ActiveSaga. The game runs directly on the Meta Quest headset as an APK and does not require a PC during gameplay.

The system combines physical movement, immersive VR gameplay, progression mechanics, and backend synchronization. The final version includes two playable game modes: Run Game and Fight Game. In both modes, the player performs real physical actions such as running in place, jumping, squatting, dodging, and attacking.

ActiveSaga also includes login and registration, saved player progress, XP, coins, levels, daily missions, weekly progress, pause functionality, help screens, audio feedback, and end-game results. After a gameplay session ends, the Unity client sends the session data to the backend server, which updates the player profile in MongoDB.

Overall, the project achieved its main purpose: turning physical activity into an interactive VR game experience that motivates the player through gameplay, feedback, and long-term progress.

6.2 Project Measures and Success Criteria

The project was evaluated according to several success criteria defined during development.

Success Criterion	Result
The game runs as a standalone application on Meta Quest	Achieved
The system includes two playable game modes	Achieved
The game detects running, jumping, and squatting in real time	Achieved after tuning
The player can register, log in, and load saved progress	Achieved
Game results are saved through the backend and MongoDB	Achieved
XP, coins, levels, daily missions, and weekly progress are updated correctly	Achieved
The game includes VR-friendly menus, help screens, pause, and results screens	Achieved
The final system can be demonstrated on real VR hardware	Achieved

Based on these criteria, ActiveSaga met the main functional and technical goals of the project. Some features, such as a full shop system and advanced upgrades, were left as future work in order to keep the final scope realistic and focus on a stable working system.

6.3 Achievement of Project Goals

The first goal was to create a VR game that encourages physical activity. This was achieved through two game modes that require real body movement. The Run Game focuses on cardio activity, while the Fight Game focuses on reaction, combat, and body-control actions.

The second goal was to detect physical movements in real time. This was achieved through a custom movement recognition system that uses the Meta Quest headset and controllers, without external body sensors.

The third goal was to implement motivational mechanisms that support repeated use. This was addressed through XP, coins, levels, daily missions, weekly rewards, and end-game results. These systems provide immediate feedback after each session and are designed to encourage the player to return to the game. However, long-term retention was not formally measured and remains a topic for future evaluation.

The fourth goal was to save player progress between sessions. This was achieved through a backend server connected to MongoDB. The backend manages authentication, player profiles, reward calculation, daily missions, weekly progress, and game-session results.

The fifth goal was to create a complete working system that can be demonstrated on real VR hardware. This was achieved by building and testing the final version as a standalone APK on the Meta Quest headset.

6.4 Known Limitations

Although the final system is functional and meets the main goals of the project, several limitations remain.

First, the game requires a safe physical play area. Since the player performs running-in-place, jumping, squatting, dodging, and arm movements, the user must have enough space to move safely.

Second, the movement recognition system does not use external leg sensors or full-body tracking. This makes the game more accessible, but it also means that some lower-body movements are inferred from headset movement rather than measured directly.

Third, height calibration affects detection accuracy. If the player does not stand naturally during calibration, squat and jump detection may be less accurate.

Fourth, the game depends on an internet connection for authentication, progress saving, daily missions, weekly progress, and backend synchronization. If the backend is unavailable, these online features cannot work correctly.

Finally, ActiveSaga is not a medical or personalized fitness application. The system encourages physical activity through gameplay, but it does not replace a professional training plan or medical recommendation.

6.5 Lessons Learned

During the project, we learned that VR fitness development requires a balance between gameplay, responsiveness, physical comfort, and technical reliability. Features that work inside the Unity Editor must still be tested on the actual Meta Quest headset, because real movement, controller input, and user comfort can only be evaluated properly in VR.

Another important lesson was that small usability issues can strongly affect the player experience. For example, unreliable button interaction in VR made the menus less comfortable to use, so the interaction areas were adjusted and made more forgiving.

We also learned the importance of separating responsibilities between the Unity client and the backend. Unity handles real-time gameplay and movement recognition, while the backend manages authentication, rewards, missions, weekly progress, and persistent data. This separation made the system more organized and easier to maintain.

Finally, the project showed the importance of scope management. Instead of adding many unfinished features, the team focused on completing and stabilizing the core system. This decision improved the quality of the final product.

6.6 Future Work

Future versions of ActiveSaga can include a full shop system, allowing players to use coins to buy cosmetic items, equipment, or visual upgrades. An upgrade or skill system can also be added, giving players more ways to customize their progress.

Additional game modes may also be developed, such as rhythm-based workouts, timed challenges, new boss fights, and multiplayer gameplay. A multiplayer mode could allow several players to train together, compete in challenges, or cooperate against enemies in a shared VR environment. This feature could make the game more social and increase long-term motivation.

6.7 Final Conclusion

ActiveSaga demonstrates how Virtual Reality can turn physical activity into an engaging and motivating game experience. The final system combines real-time movement recognition, two playable game modes, player progression, daily and weekly motivation systems, backend synchronization, and a standalone Meta Quest build.

The project achieved its main objectives and provided practical experience in VR development, Unity, C#, backend development, MongoDB, REST API communication, authentication, deployment, and user-centered testing. Overall, ActiveSaga shows how gameplay, fitness motivation, and real-time interaction can be combined into one complete software engineering project.

7.References

[1] Unity Technologies. Unity Manual: XR

[2] Meta. Develop Unity apps for Meta Quest VR headsets

[3] Express.js. Express - Node.js web application framework

[4] MongoDB. MongoDB Documentation

[5] Mongoose. Schemas Documentation

[6] IETF. RFC 7519: JSON Web Token (JWT)

[7] npm. bcrypt package documentation

[8] Render. Deploy a Node Express App on Render

[9] World Health Organization. Physical Activity Fact Sheet

[10] Shameli, A., Althoff, T., Saberi, A., and Leskovec, J. How Gamification Affects Physical Activity: Large-scale Analysis of Walking Challenges in a Mobile Application

[11] Is Playing Exergames Really Exercising? A Meta-Analysis of Energy Expenditure in Active Video Games.Cyberpsychology, Behavior, and Social Networking

[12] Faric, N., Potts, H. W. W., Hon, A., Smith, L., Newby, K., Steptoe, A., & Fisher, A. (2019).
What Players of Virtual Reality Exercise Games Want: Thematic Analysis of Web-Based Reviews

[13] Ryan, R. M., & Deci, E. L. (2000). Self-Determination Theory and the Facilitation of Intrinsic Motivation, Social Development, and Well-Being

[14] Sailer, M., Hense, J. U., Mayr, S. K., & Mandl, H. (2017). How Gamification Motivates: An Experimental Study of the Effects of Specific Game Design Elements on Psychological Need Satisfaction

Appendix A: User Guide - Operating Instructions

A.1 Purpose

This guide explains how to use ActiveSaga from the player's point of view. It describes the normal user flow, from launching the game and logging in, through selecting a game mode, playing a session, viewing the results, and returning to the main scene.

ActiveSaga is designed for the Meta Quest headset and uses the VR headset and controllers to detect the player's physical movements during gameplay.

A.2 Required Equipment and Conditions

Before starting the game, the user should make sure that:

- The Meta Quest headset is charged and ready to use
- Both VR controllers are connected
- The headset is connected to the internet
- The player has enough safe space to move, jump, squat, dodge, and swing the controllers

An internet connection is required because ActiveSaga uses online login, player profile loading, progress saving, missions, weekly progress, and backend synchronization.

A.3 Launching the Game

When the user launches ActiveSaga on the Meta Quest headset, the default first screen is the authentication screen. From this screen, the user can either log in with an existing account or create a new one if no account is available.

During registration, all fields are required. If any field is missing or invalid, the system displays an error message in red and the registration cannot continue until the issue is fixed.

All account and player data are stored in the database and loaded from it after a successful login or registration.

In addition, if the player has logged in within the last 30 days and did not log out, a saved authentication token is used, so the player skips this screen and is taken directly to the next screen.

Figure A.1: Login screen for existing ActiveSaga users

Figure A.2: Registration screen for creating a new ActiveSaga account

A.4 Main Scene Overview

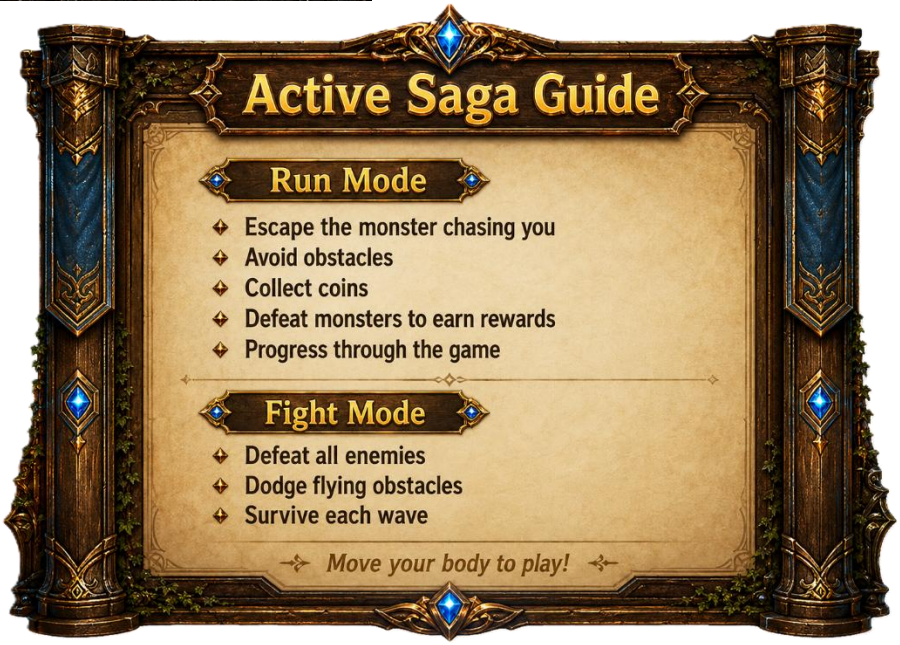
After login, the player enters the main scene. This scene contains the main game screens and allows the player to review progress, check goals, and choose a gameplay session.

The main scene includes the following screens:

Welcome:

Welcome Screen

The Welcome Screen greets the player by name and provides access to the game guide and logout option



ActiveSaga Guide

The ActiveSaga Guide explains the main objectives of Run Mode and Fight Mode and introduces the player to the required physical actions

Daily Missions:

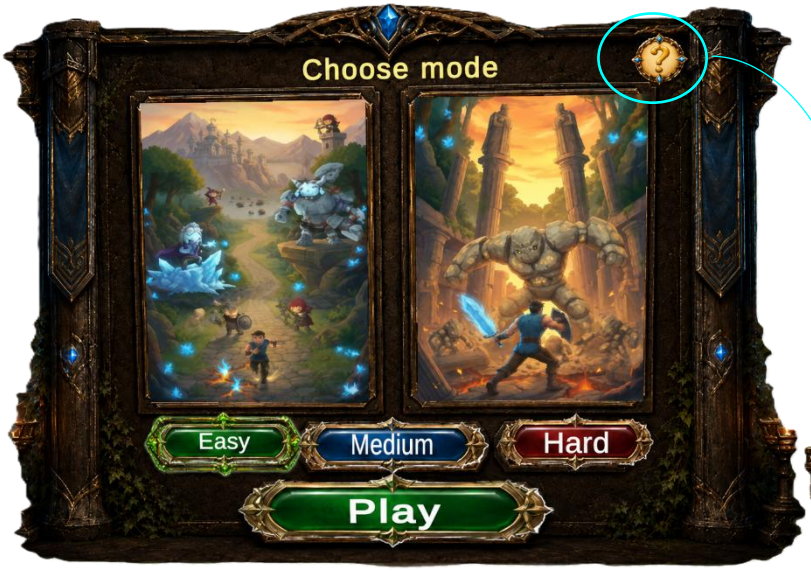
Daily Missions Screen

The Daily Missions Screen displays the player's available missions, their objectives, and the XP and coin rewards for completing them

Daily Missions Guide

The Daily Missions Guide explains that missions provide additional rewards and must be completed within a single gameplay session

Game Mode Selection:



Game Mode Selection Screen

The Game Mode Selection Screen allows the player to choose between Run Mode and Fight Mode, select a difficulty level, and start the game

ActiveSaga Guide

The ActiveSaga Guide presents the main objectives and physical challenges of both Run Mode and Fight Mode

Progress and Achievements:

Progress and Achievements Screen

The Status Screen displays the player's current level, XP progress, and tracked activity statistics, including active time, distance, and jumps

Status Guide

The Status Guide explains how the player earns XP, progresses through levels, and tracks overall physical activity

Weekly Streak:

Daily Streak Screen

The Daily Streak Screen displays the player's weekly activity progress and the accumulated play time toward the daily five-minute goal

Daily Streak Guide

The Daily Streak Guide explains that five active days unlock a weekly reward and that the required daily play time can be completed across multiple sessions

A.5 Selecting a Game Mode

To start playing, the player first reviews the available missions and goals in the main scene. Then, the player selects one of the two game modes:

Run Game: A movement-based running mode focused on cardio activity

Fight Game: A combat and reaction mode focused on attacking enemies, dodging, and avoiding incoming obstacles

After choosing a game mode, the player selects a difficulty level. The available difficulty levels are Easy, Medium, and Hard. After the mode and difficulty are selected, the game session begins.

A.6 Playing the Run Game

In the Run Game, the player moves forward by running in place. This mode is designed mainly as an aerobic activity, requiring continuous physical movement throughout the session. A monster chases the player from behind, and the player must keep moving to escape and avoid being caught.

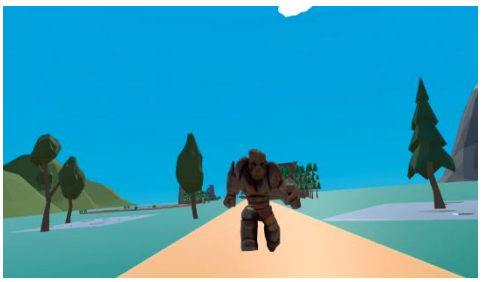
The Run Game includes three difficulty levels: Easy, Medium, and Hard. Each level features a different visual environment and provides a distinct gameplay experience.

The main actions in this mode are:

- Running in place to move forward
- Jumping over obstacles
- Squatting under obstacles
- Attacking monsters that appear during the run

The main goal of the Run Game is to provide an aerobic workout by encouraging continuous movement while the player escapes the chasing monster, avoids obstacles, progresses through the environment, and completes the session.





A.7 Playing the Fight Game

In the Fight Game, the player faces enemies and incoming threats. The main goal is to defeat as many enemies as possible while avoiding obstacles and attacks.

The Fight Game includes three difficulty levels: Easy, Medium, and Hard. Unlike the Run Game, the visual environment remains the same, while the difficulty changes according to the number and frequency of enemies and obstacles that appear.

The main actions in this mode are:

- Swinging the controllers to attack enemies
- Dodging incoming obstacles and threats
- Jumping when required
- Moving physically in response to gameplay events
- Continuing to move in place between attacks to gain additional energy
- Catching coins during waiting periods to earn extra coins

The Fight Game focuses on reaction speed, continuous movement, and combat. The player must remain active between attacks to maintain energy, collect additional rewards, and survive each wave. The session continues until the player completes the game, loses, or exits manually.

A.8 Pausing or Stopping a Game Session

During gameplay, the player can pause the session. The pause option allows the player to stop the gameplay temporarily and then continue or exit the session.

From the pause menu, the player can resume the game or stop the current session and return to the main flow.

A.9 Results Screen

At the end of each gameplay session, the player reaches the results screen. This screen summarizes the performance of the current session and shows the rewards earned by the player.

The results screen includes:

- Time played
- XP earned
- Coins earned
- Current level
- XP progress
- Distance completed
- Jumps performed
- Enemies defeated
- Additional mode-specific statistics

After the results are displayed, the session data is saved through the backend. The player can then return to the main scene and see the updated progress in the progress screen, missions screen, and weekly streak screen.

A.10 Returning to the Main Scene

After the session ends and the results are shown, the player returns to the main scene. From there, the player can review the updated progress, check whether missions were completed, see the weekly streak status, and start another gameplay session.

This completes the normal user flow of ActiveSaga: launch the game, log in or register, review goals, choose a game mode and difficulty, play a session, view results, save progress, and return to the main scene.

Appendix B: Maintenance Guide

B.1 Purpose

This maintenance guide is intended for developers or project maintainers who may continue working on ActiveSaga after the project submission. The guide explains the system structure, development environment, backend configuration, Unity client configuration, deployment process, and common maintenance tasks.

The goal of this guide is to make it easier to update, fix, extend, and redeploy the system in the future.

B.2 System Overview for Maintainers

ActiveSaga is built as a client-server system.

The **Unity client** runs directly on the Meta Quest headset. It is responsible for the VR experience, gameplay logic, movement recognition, UI screens, audio, pause system, help screens, and results display.

The **backend server** is implemented using Node.js and Express. It is responsible for user authentication, player profile loading, XP, coins, levels, daily missions, weekly progress, reward calculation, and saving gameplay results.

The **MongoDB database** stores persistent data such as user accounts, player profiles, progress, statistics, missions, and weekly activity.

The Unity client communicates with the backend using REST API requests. The client does not access MongoDB directly. All database updates are performed through the backend server.

B.3 Required Software, Hardware and Accounts

Before cloning and opening the project, the maintainer should prepare the required development environment. The project was developed and tested on Windows, and the following instructions assume a Windows development environment.

Required Hardware

- A Windows development computer capable of running Unity
- A Meta Quest headset
- Two Meta Quest controllers
- A USB-C data cable that supports data transfer
- A stable internet connection

Required Software

- Git, for cloning and maintaining the repository
- Unity Hub
- Unity Editor version 6000.3.13f1
- Android Build Support for Unity
- Android SDK and NDK Tools
- OpenJDK
- Node.js version 20.19.0 or later
- npm, which is installed together with Node.js

- A code editor such as Visual Studio Code
- The Meta Horizon mobile application
- Oculus ADB Drivers for Windows, which allow the computer to detect and communicate with the Meta Quest headset during development

Required Accounts and Access

- A Unity account for installing and using Unity Editor
- A GitHub account with write access when pushing maintenance changes. The public repository can be cloned without write access
- Access to the MongoDB Atlas project when maintaining the database
- Access to the Render service when maintaining or redeploying the backend
- A verified Meta developer account that belongs to a Meta developer team or organization

The current project owner should grant the maintainer the required GitHub, MongoDB Atlas, Render, and Meta developer permissions before maintenance work begins. Passwords, connection strings, secret keys, and other sensitive values must be transferred through a secure channel and must not be stored in this document.

Installing Git and Node.js

1. Download and install Git
2. Open PowerShell and verify that Git was installed successfully:
   git --version
3. Download and install Node.js version 20.19.0 or later
4. Open PowerShell and verify the Node.js and npm installations:
   node --version
   npm --version
5. Confirm that all three commands display valid version numbers

Installing Unity

1. Download and install Unity Hub
2. Open Unity Hub and sign in with a Unity account
3. Open the **Installs** tab
4. Select **Install Editor**
5. Install Unity Editor version **6000.3.13f1**
6. During installation, select **Android Build Support**
7. Under Android Build Support, also select:
   - Android SDK and NDK Tools
   - OpenJDK
8. Complete the installation and verify that Unity version 6000.3.13f1 appears in the Unity Hub Installs list

These Android modules are required for building ActiveSaga as a standalone Meta Quest APK.

Preparing the Meta Quest Development Environment

1. Install the Meta Horizon mobile application and pair it with the Meta Quest headset.
2. Confirm that the Meta account is registered and verified as a developer account.
3. Confirm that the account belongs to a Meta developer team or organization.
4. Install the Oculus ADB Drivers on the Windows development computer.
5. Restart the computer if required after installing the drivers.

The headset does not need to remain connected while editing normal Unity scripts. However, it must be correctly configured and connected when testing VR interaction or building and installing an APK.

B.4 Cloning the Repository

The complete ActiveSaga source code is stored in the following GitHub repository:

https://github.com/DanielChernov99/ActiveSaga

To download the project, open a terminal or PowerShell window and run:

git clone https://github.com/DanielChernov99/ActiveSaga.git

After the clone operation is completed, enter the project directory:

cd ActiveSaga

The repository contains two main project phases:

- Part1 contains the earlier proof-of-concept and project documentation.
- Part2 contains the final runnable ActiveSaga system.

The final system is divided into two main folders:

- Part2/ActiveSaga-Game : the Unity VR client
- Part2/ActiveSaga-API : the Node.js and Express backend

All development, maintenance, building, and deployment operations for the final system should be performed using the projects located under Part2.

B.5 Opening and Restoring the Unity Project

To open the Unity project:

1. Start Unity Hub
2. Select Add or Add Project from Disk
3. Browse to the cloned repository
4. Select the following folder: Part2/ActiveSaga-Game
5. Confirm that Unity version 6000.3.13f1 is selected
6. Open the project
7. Wait until Unity finishes importing the assets and restoring the required packages
8. Check the Console window and make sure that there are no compilation errors

The project may take several minutes to open for the first time because Unity must generate the Library folder and import the project assets.

The maintainer should not select the repository root or the Part2 folder as the Unity project. The correct Unity project folder is ActiveSaga-Game, which contains the Assets, Packages, and ProjectSettings directories.

After the project opens, the main scenes can be found under:

Assets/ActiveSaga/Scenes

The main scenes are:

- Login
- Main New
- Run Game
- Fight Game

B.6 Running the Backend Locally

The final standalone APK normally communicates with the deployed backend on Render. Therefore, running the backend locally is not required when testing the production configuration.

A local backend is used when modifying or debugging the server, or when testing the Unity client against local backend changes.

To run the backend locally, open a terminal and navigate to:

cd Part2/ActiveSaga-API

Install the backend dependencies:

npm install

Create a file named .**env** inside the ActiveSaga-API directory.

The file should contain:

MONGO_URI=<MongoDB connection string>

JWT_SECRET=<private JWT secret>

PORT=3000

The actual credentials must not be written in the maintenance document or committed to GitHub.

Start the backend:

node server.js

A successful startup should display messages indicating that the server connected to MongoDB and started on the configured port.

To verify that the server is running, open the following address in a browser:

http://localhost:3000/

The server should return a response indicating that the ActiveSaga API is running.

To stop the local server, return to the terminal and press Ctrl+C

B.7 API Configuration in Unity

The production version of ActiveSaga does not require a locally running backend server. The standalone APK installed on the Meta Quest headset communicates with the deployed backend server on Render.

The deployed backend base URL is:

https://active-saga-api.onrender.com

The Unity client uses several scripts for authentication, player-data requests, and completed game-session submission:

- AuthManager
- PlayerApiService
- ApiGameResultSubmitter

Each component uses a different form of the backend URL. Therefore, the maintainer must configure each field according to the following table:

Component	Field	Production value
AuthManager	authBaseUrl	https://active-saga-api.onrender.com/api/auth
AuthManager	playerBaseUrl	https://active-saga-api.onrender.com/api/player
PlayerApiService	playerApiBaseUrl	https://active-saga-api.onrender.com/api/player
ApiGameResultSubmitter	baseUrl	https://active-saga-api.onrender.com
ApiGameResultSubmitter	completeGameEndpoint	/api/player/complete-game-session

The ApiGameResultSubmitter base URL must not include /api/player, because the complete game-session endpoint is stored separately and is appended to the base URL when the request is created.

These URLs allow the Meta Quest application to authenticate users, validate saved login tokens, load player data, retrieve missions and weekly progress, submit completed game sessions, and save updated progress in MongoDB.

For local development in the Unity Editor, the corresponding URLs may be changed to:

Component	Field	Local value
AuthManager	authBaseUrl	http://localhost:3000/api/auth
AuthManager	playerBaseUrl	http://localhost:3000/api/player
PlayerApiService	playerApiBaseUrl	http://localhost:3000/api/player
ApiGameResultSubmitter	baseUrl	http://localhost:3000
ApiGameResultSubmitter	completeGameEndpoint	/api/player/complete-game-session

When testing from the Meta Quest headset while the backend runs locally on the developer's computer, localhost must not be used. On the headset, localhost refers to the headset itself rather than the development computer.

Instead, the maintainer must use the computer's local network IP address. For example:

http://192.168.x.x:3000

The computer and the Meta Quest headset must be connected to the same local network. The local IP address can be found by running:

ipconfig

The maintainer must also verify that the operating-system firewall allows the backend to receive connections on port 3000.

Before creating the final APK, all API values must be restored to the deployed Render URLs shown in the production configuration table.

For serialized URL fields, the maintainer should verify the values stored in the Unity Inspector, because scene or prefab values may override the default values written in the C# scripts.

B.8 Database Maintenance

MongoDB stores the persistent data of the system.

The database includes player-related data such as:

- User accounts
- Hashed passwords
- Player profiles
- XP and coins
- Levels
- Total play time
- Total distance
- Daily missions
- Weekly progress
- Gameplay statistics

Maintainers should avoid editing production data manually unless necessary. If manual changes are required, they should be performed carefully through MongoDB tools.

Important database maintenance notes:

- Do not store plain text passwords
- Do not allow the Unity client to access MongoDB directly
- Do not expose the MongoDB connection string in the Unity project
- Do not commit database credentials to GitHub
- Use test users when checking new features
- Before changing database schemas or models, check that the backend logic and Unity response handling still match the updated data structure

B.9 Render Deployment Maintenance

The backend is deployed using Render.

To update the deployed backend:

- Push the backend changes to GitHub
- Open the Render service dashboard
- Check that the correct GitHub repository and branch are connected
- Make sure all environment variables are defined in Render
- Deploy the latest version
- Check the Render logs after deployment
- Test login, profile loading, and game-session submission from the Unity client

If the deployed backend URL changes, the URL must be updated inside the Unity API service scripts, and a new APK build should be created.

B.10 Building a New APK

Before building the project, verify that Unity Editor version 6000.3.13f1 and the required Android modules are installed.

Enabling Developer Mode on the Meta Quest Headset

Developer Mode must be enabled before Unity can install and run development builds on the headset.

1. Confirm that the Meta account is verified and belongs to a Meta developer team or organization.
2. Open the Meta Horizon mobile application.
3. Select the headset icon.
4. Select the paired Meta Quest headset.
5. Open **Headset Settings**.
6. Select **Developer Mode**.
7. Enable Developer Mode.
8. Restart the headset if required.

Developer Mode usually needs to be enabled only once for each headset.

Connecting the Headset to the Computer

1. Connect the Meta Quest headset to the development computer using a USB-C data cable.
2. Put on the headset.
3. Open the headset settings and verify that development-related USB notifications are enabled.
4. When the USB debugging request appears, select **Always allow from this computer**.
5. Confirm the USB debugging request.

If the USB debugging request does not appear, disconnect and reconnect the cable, verify that Developer Mode is enabled, and make sure that the cable supports data transfer rather than charging only.

Preparing the Unity Build

1. Open the ActiveSaga Unity project through Unity Hub.
2. Wait until Unity finishes importing all assets and packages.
3. Check the Unity Console and verify that there are no red compilation errors.
4. Open:
   File > Build Profiles
5. Under the available platforms, select **Meta Quest**.
6. If the platform is not active, select **Enable Platform** or **Switch Platform**, according to the option displayed by Unity.
7. Wait until Unity finishes switching and preparing the platform.

Verifying the Build Scenes

Before building, verify that the following scenes are enabled in this exact order:

1. Login
2. Main New
3. Run Game
4. Fight Game

The SampleScene scene should not be included in the final build.

If a new game mode or required scene is added in the future, it must also be added to the Build Profiles scene list.

Verifying the Backend Configuration

Before creating the final APK, verify that all API fields use the deployed Render production values described in Section B.7.

In particular, verify the values used by:

- AuthManager
- PlayerApiService
- ApiGameResultSubmitter

The final APK must not use localhost or the development computer's local network IP address.

The maintainer should verify both the values defined in the C# scripts and the serialized values stored in the Unity Inspector, because scene or prefab values may override script default values.

Building and Installing the APK

1. In the active Meta Quest Build Profile, locate the **Run Device** field.
2. Select the connected Meta Quest headset.
3. If the headset does not appear, select **Refresh**.
4. Verify that the headset remains connected and that USB debugging was approved.
5. Select **Build and Run**.
6. Choose a folder in which to save the APK.
7. Enter an appropriate APK filename, for example:
   ActiveSaga.apk
8. Select **Save**.
9. Wait until Unity completes the build process.
10. Unity should install the APK on the connected headset and launch the application automatically.
11. Put on the headset and verify that ActiveSaga starts correctly.

The APK file should be stored outside Unity temporary folders. Build files should not normally be committed to GitHub unless they are explicitly required for a release.

Testing the New APK

After building and installing a new APK, the maintainer should test the complete application flow:

- The application launches without requiring a PC connection
- Registration works with a new test account
- Login works with an existing test account
- Saved player data loads correctly
- The main scene opens correctly
- The Daily Missions screen displays the current missions
- The Weekly Streak screen displays the current progress
- The Run Game starts, operates, and ends correctly
- The Fight Game starts, operates, and ends correctly
- Movement recognition works on the real headset
- Controller interaction works correctly
- Pause and resume work correctly
- The results screen displays the session information
- The completed session is submitted to the backend
- XP, coins, missions, weekly progress, and player statistics are updated
- The updated progress appears after returning to the main scene

If the Headset Is Not Detected

If the Meta Quest headset does not appear in the Unity Run Device list:

- Verify that Developer Mode is enabled
- Verify that the Meta account is registered as a developer
- Verify that the Oculus ADB Drivers are installed
- Use a USB-C cable that supports data transfer
- Disconnect and reconnect the headset
- Approve the USB debugging request inside the headset
- Select **Refresh** in the Unity Run Device list
- Restart Unity after connecting the headset
- Restart the headset and development computer if the problem continues

B.11 Common Maintenance Tasks

Changing Reward Values

Reward values such as XP, coins, daily mission rewards, and weekly rewards should be updated in the backend logic. After changing reward logic, the maintainer should test a full game session and verify that the results screen and database values are updated correctly.

Changing Daily Missions

Daily mission requirements should be updated in the backend mission logic. The maintainer should verify that missions are completed only when the required goal is achieved in the same gameplay session.

Changing Weekly Progress Rules

Weekly progress rules should be updated in the backend weekly progress logic. For the current version, the player must play at least five minutes in a day to complete the daily activity requirement. Completing five active days in the same week gives the player a larger weekly reward.

Changing Movement Detection

Movement detection should be updated in the Unity scripts. Any changes to running, jumping, squatting, dodging, or attacking thresholds must be tested on the Meta Quest headset. Small changes in thresholds may strongly affect the player experience.

Adding a New Game Mode

To add a new game mode, the maintainer should update both the Unity client and the backend if the new mode requires new session statistics. The Unity client should include the new scene, gameplay logic, UI connection, and results handling. The backend should support the new result fields and save the relevant statistics.

Adding New UI Screens

New UI screens should be added inside Unity. The maintainer should check that the screen is readable in VR, that the controller interaction works correctly, and that the screen does not block the player's movement or view.

B.12 Troubleshooting

Login or Registration Does Not Work

- Check that the headset is connected to the internet
- Check that the backend server is running
- Check that the API URL in Unity is correct
- Check the Render logs or local server console
- Check that MongoDB is connected
- Check that the required environment variables are configured

Progress Is Not Saved

- Check that the game-session result is sent to the backend
- Check that the user token is included in the request
- Check that the backend receives the request without authentication errors
- Check that MongoDB updates the player profile correctly
- Check that Unity handles the backend response correctly

Buttons Do Not Respond in VR

- Check the VR ray interaction setup
- Check the Canvas configuration
- Check that the buttons have correct interaction areas
- Check that the controllers are connected
- Test the screen inside the headset, not only in the Unity Editor

Movement Detection Feels Incorrect

- Repeat height calibration
- Check the movement detection thresholds
- Test on the real headset
- Verify that running, jumping, squatting, dodging, and attacking are not triggering each other incorrectly

APK Build Fails

- Check that the correct build platform is selected
- Check that Android build support is installed
- Check that all required scenes are included
- Check for missing scripts or broken references
- Check Unity console errors before building again

B.13 Version Control Guidelines

The project should be maintained using Git and GitHub.

Recommended version control rules:

- Commit small and meaningful changes
- Write clear commit messages
- Do not commit .env files
- Do not commit secret keys or database credentials
- Do not commit large build outputs unless required
- Do not commit Unity temporary folders such as Library, Temp, Logs, and Obj
- Keep the deployed backend branch stable
- Test important changes before pushing them to the main branch

B.14 Maintenance Summary

ActiveSaga is designed so that the Unity client handles real-time VR gameplay, while the backend handles authentication, rewards, missions, weekly progress, and persistent data. This separation allows the system to remain responsive on the Meta Quest headset while keeping important player data managed through the server and database.

Future maintainers should focus on testing changes on the real headset, keeping backend credentials secure, checking API communication after every major change, and updating both the Unity client and backend when new gameplay data is added.